

Plain2MeTTa v2 Public Evaluation Workbench

Evaluation report · 2026-08-16

Repository: `bgoertzel-sing/plain2metta`

Branch: `agent/plain2metta-public-evaluation-ui`

Implementation commit: `6b9b44e`

Outcome

The workbench takes a concrete Plain specification through immutable content-addressed artifacts, exact reviewed snapshots, logical IR and findings, deterministic reference generation, an explicit sandbox handoff, bounded Python execution, and a joined traceability report. The browser exposes each stage for inspection.

Truthful validation labels: MeTTa is generated and parenthesis-syntax-checked, but not runtime-validated. Python is labeled sandbox-executed only when the recorded subprocess exits successfully. No LLM/provider is invoked.

Architecture

Stage	Implementation and invariant
Input / elaboration	Bounded 128 KiB input; deterministic elaboration retains exact source and IDs.
Review	<code>Phase3ReviewLog</code> decisions bind exact elaborated/test hashes; reviewed snapshots are immutable.
Logical IR	Typed <code>LogicalIRDocument</code> , source-clause provenance, planned tests, hash-bound findings, explicit approval.
Generation	<code>CompilerOutputBundle</code> with safe relative paths and per-file spec/test IDs; stored inert before approval.
Sandbox	Separate adapter writes to a temporary directory and runs Python <code>-I</code> with CPU, address-space, file-size, descriptor, process-count, and wall-clock limits.
Traceability	Joins exact output and result artifacts; each requirement is passing, failing, skipped, or untested.

Design choices

- Built from v2 commit `590fc7d`, not the pre-v2 Flask branch.
- Kept the core provider-free and injected no credential or network integration.
- Used a deterministic generator so reviewers can distinguish pipeline behavior from model quality.
- Preserved fail-closed strict request schemas and the existing approval/invalidation model.
- Bound deployment only to the exact Tailnet address; no Funnel, wildcard LAN bind, sudo, or security change.

Evidence

Gate	Observed result
Focused evaluation/API/v2	6 / 6 passed
Full provider-free suite	655 / 655 passed in 11.960 s
Static compilation	<code>compileall</code> passed
Diff hygiene	<code>git diff --check</code> passed
Browser smoke	GET / → HTTP 200
API/sandbox smoke	POST /api/evaluate → exit 0, 12 artifacts

Primary source locations: `src/specatom_hs/evaluation.py`, `src/specatom_hs/evaluation_sandbox.py`, `webapp/app.py`, `webapp/templates/index.html`, and `tests/test_evaluation*.py`.

Limitations and security boundary

This is an evaluation generator, not a production compiler. MeTTa runtime semantics are not tested. The Python subprocess uses resource limits and a capability-minimal generated program, but the adapter does not claim a kernel network namespace, seccomp profile, container boundary, or adversarial-code isolation. It must therefore remain restricted to the deterministic generator until a stronger external sandbox is integrated. The tmux deployment survives shell disconnects but not host reboot.

Operation

Tailnet URL: `http://100.72.218.34:8081/`

Inspect: `tmux attach -t plain2metta-eval`

Stop: `tmux kill-session -t plain2metta-eval`

Restart: `tmux new-session -d -s plain2metta-eval 'cd .../plain2metta-public-evaluation-ui && PYTHONPATH=src python3 webapp/app.py --host 100.72.218.34 --port 8081'`

Generated reproducibly from this HTML with WeasyPrint. Artifact hashes are recorded beside the run ledger.